```
"use strict";
const test = require("node:test"), assert = require("node:assert/strict");
const { FV, fakeDb } = require("./fakes");
const { createProvider, ProviderError } = require("../lib/esim/provider"),
makeCatalog = require("../lib/esim/catalog"), makeFulfillment =
require("../lib/esim/fulfillment"), createDemo = require("../lib/esim/demoProvider");
const makeRewards = require("../lib/rewards"), { eraseUser } =
require("../lib/erasure"), { safePath, errorHandler } =
require("../middleware/errors");
const L = { warn() {}, error() {} };
const prov = (resp) => createProvider({ baseUrl: "https://api.example.test/x",
accessCode: "ACCESS-1234", signingKey: "KEY-12345", fetchImpl: async () => { if (resp
instanceof Error) throw resp; return { status: resp.s ?? 200, json: async () => { if
(resp.bad) throw new Error("x"); return resp.b; } }; } });

test("commande fournisseur : toute réponse ambiguë = résultat INCONNU ; seul un refus
certain est « rejected »", async () => {
  const order = (p) => p.orderProfiles({ transactionId: "o1", packageCode: "C",
units: 100 });
  for (const resp of [{ s: 200, bad: true }, { s: 200, b: { success: true, obj: {
orderNo: "../x" } } }, { s: 200, b: { success: true, obj: {} } }, { s: 200, b: "x" },
{ s: 503, b: {} }, new Error("net")]) await assert.rejects(() => order(prov(resp)), {
kind: "unknown" });
  await assert.rejects(() => order(prov({ s: 200, b: { success: false, errorCode:
"310" } })), { kind: "rejected" }); await assert.rejects(() => order(prov({ s: 401,
b: {} })), { kind: "auth" });
  assert.deepEqual(await order(prov({ s: 200, b: { success: true, obj: { orderNo: "B-
26_04" } } })), { orderNo: "B-26_04" });
});
async function shop(fail) {
  const db = fakeDb(), t = { v: 1e6 }, provider = createDemo({ fail }), cfg = { esim:
{ planIdSecret: "s".repeat(40), marginBps: 1500 } };
  const catalog = makeCatalog({ provider, config: cfg, log: L, now: () => t.v }), f =
makeFulfillment({ db, FV, catalog, provider, log: L, now: () => t.v });
  await db.collection("users").doc("u1").set({ balance: 10, points: 0 }); return {
db, f, plan: (await catalog.plansFor("HT"))[0] };
}
test("achat : une erreur « invalide » du fournisseur ne rembourse JAMAIS
automatiquement (risque de perte)", async () => {
  const s = await shop("invalid"); const r = await s.f.purchase({ uid: "u1", planId:
s.plan.id }); assert.equal(r.code, "PENDING"); assert.equal(s.db._data.get("orders/"
+ r.orderId).status, "unknown");
  assert.equal(+s.db._data.get("users/u1").balance.toFixed(2), +(10 -
s.plan.price).toFixed(2)); assert.ok(!s.db._data.has("refunds/" + r.orderId));
  for (const kind of ["rejected", "auth"]) { const x = await shop(kind); const y =
await x.f.purchase({ uid: "u1", planId: x.plan.id }); assert.equal(y.code,
```

```
"PROVIDER_FAILED"); assert.equal(x.db._data.get("users/u1").balance, 10); }
});
test("journal : le jeton du webhook (dans l'URL) est masqué partout", () => {
  assert.equal(safePath("/api/webhooks/esim/SECRETTOKEN-abc_123"),
"/api/webhooks/esim/[masqué]"); assert.equal(safePath("/api/esims/o1/qr.svg"),
"/api/esims/o1/qr.svg");
  const logged = []; const h = errorHandler({ error: (o) => logged.push(o) }); h(new
Error("x"), { id: "r", path: "/api/webhooks/esim/SECRETTOKEN-abc_123" }, {
headersSent: false, status() { return this; }, json() {} }, () => {});
  assert.ok(!JSON.stringify(logged).includes("SECRETTOKEN"));
});
test("parrainage : achat trop petit = pas de récompense et pas de « première
activation » consommée ; un achat qualifiant récompense ensuite", async () => {
  const db = fakeDb(), r = makeRewards({ db, FV, config: { referralCents: 300,
referralMax: 5, referralMinSpendCents: 500 }, now: () => 1e12 });
  await db.collection("users").doc("ref").set({ balance: 0, suspended: false });
await db.collection("users").doc("kid").set({ balance: 0, referred_by: "ref" });
  assert.equal((await r.onFirstActivation("kid", "o1", 276)).code, "BELOW_MIN");
assert.equal(db._data.get("users/kid").first_active_at, undefined);
assert.equal(db._data.get("users/ref").balance, 0);
  assert.equal((await r.onFirstActivation("kid", "o2", 500)).code, "REWARDED");
assert.equal(db._data.get("users/ref").balance, 3); assert.equal((await
r.onFirstActivation("kid", "o3", 900)).code, "NONE");
});
test("droit à l'oubli : TOUTES les collections sont pseudonymisées, rien
d'identifiant ne reste, les autres comptes sont intacts, opération idempotente",
async () => {
  const db = fakeDb(), set = (p, d) => db.collection(p.split("/")
[0]).doc(p.split("/")[1]).set(d), U = "u1", O = "u2";
  await set("users/" + U, { email: "u1@example.com", balance: 5, profile: {
firstName: "Marie" }, referral_code: "ABCD2345" }); await set("users/" + O, { email:
"u2@example.com", balance: 9, referred_by: U });
  await db.collection("users").doc(U).collection("consents").doc("c1").set({ ip:
"10.0.0.1", version: "v" });
  await set("orders/a", { uid: U, cc: "HT", price: 2.07, margin_units: 2700, esim: {
lpa: "LPA:1$SECRET$X", iccid: "8900" } }); await set("orders/b", { uid: O, cc: "US",
esim: { lpa: "LPA:1$AUTRE$Y" } });
  await set("payments/p1", { uid: U, amount: 5, txRef: "NC123456" }); await
set("audit/x1", { by: U, action: "profile_update" }); await set("audit/x2", { by:
"admin", action: "sanction", target: U });
  await set("support_tickets/SUP-1", { user_id: U, email: "u1@example.com", subject:
"Mon dépôt", message: "mon message privé", ip_address: "10.0.0.1", attachment: {
name: "a.png" }, category: "deposit" });
  await set("downloads/d1", { user_id: U, ip_address: "10.0.0.1", platform: "android"
}); await set("promo_redemptions/BIENVENU_u1", { uid: U, code: "BIENVENU" });
  await set("referral_codes/ABCD2345", { uid: U }); await set("bonuses/" + U, { uid:
U, amount: 3 }); await set("rewards/r1", { kind: "referral", referee: U, referrer: O
}); await set("rewards/r2", { kind: "referral", referee: O, referrer: U }); await
set("rewards/r3", { kind: "points", uid: U });
  const n = await eraseUser({ db, FV, uid: U, anon: "anon_x" }); assert.ok(n > 10);
```

```
await eraseUser({ db, FV, uid: U, anon: "anon_x" });          // 2e passage : aucune
erreur
  const dump = JSON.stringify([...db._data.entries()].filter(([k]) =>
!k.startsWith("users/u2") && !k.startsWith("orders/b")).map(([, v]) => v));
  for (const leak of ['"u1"', "u1@example", "SECRET", "NC123456", "mon message",
"10.0.0.1", "8900", "Marie", "ABCD2345"]) assert.ok(!dump.includes(leak), "donnée
restante : " + leak);
  assert.deepEqual(db._data.get("users/u1"), { deleted: true, deleted_at:
db._data.get("users/u1").deleted_at, anon_id: "anon_x" });
assert.equal(db._data.has("referral_codes/ABCD2345"), false);
assert.equal(db._data.has("bonuses/u1"), false);
assert.equal(db._data.has("users/u1/consents/c1"), false);
  assert.deepEqual([db._data.get("orders/a").cc, db._data.get("orders/a").price,
db._data.get("orders/a").uid], ["HT", 2.07, "anon_x"], "comptabilité conservée, sans
identité"); assert.equal(db._data.get("audit/x2").target, "anon_x");
  assert.equal(db._data.get("users/u2").email, "u2@example.com");
assert.equal(db._data.get("users/u2").referred_by, undefined);
assert.ok(db._data.get("orders/b").esim.lpa.includes("AUTRE"), "autre compte
intact"); assert.equal(db._data.get("rewards/r1").referee, "anon_x");
assert.equal(db._data.get("rewards/r1").referrer, "u2");
});
```